

Optimizing AI & ML

*EcoTern Summer Bootcamp 2026
Jiesong Liu — PhD candidate, NCSU
CS (PICTURE group, advised by Prof.
Xipeng Shen)*

About me

3rd-year PhD in CS at NCSU, in Prof. Xipeng Shen's **P**rogramming Systems and **I**ntelligent **C**omputing for **F**uture group

*Making computing more **intelligent** and **efficient** through innovations in system software*

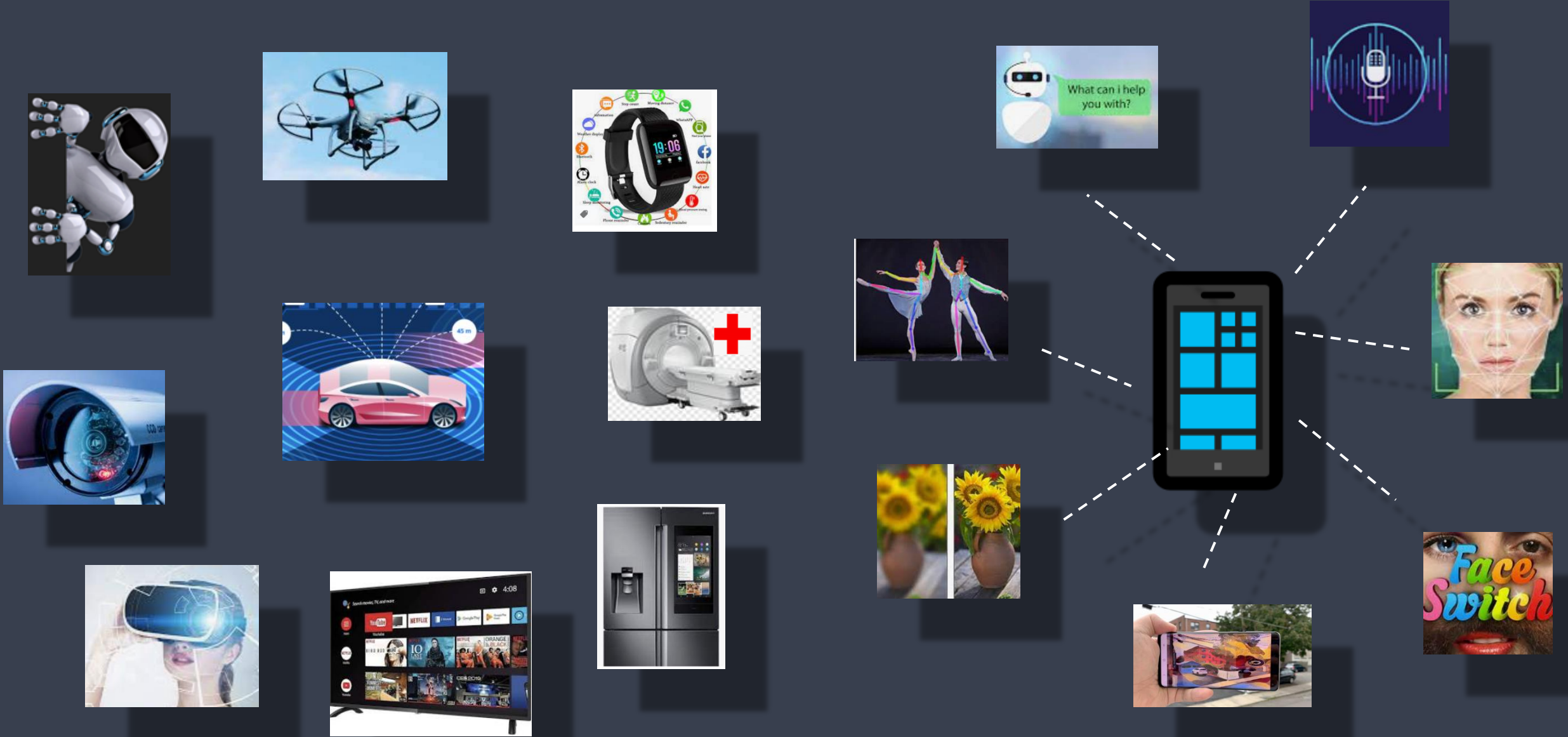
- Compilers & Code Optimizations
- High Performance Computing
- Efficient AI & Machine Learning

Real-time or **real time** describes various operations in computing or other processes that must guarantee response times within a specified time (deadline), usually a relatively short time.

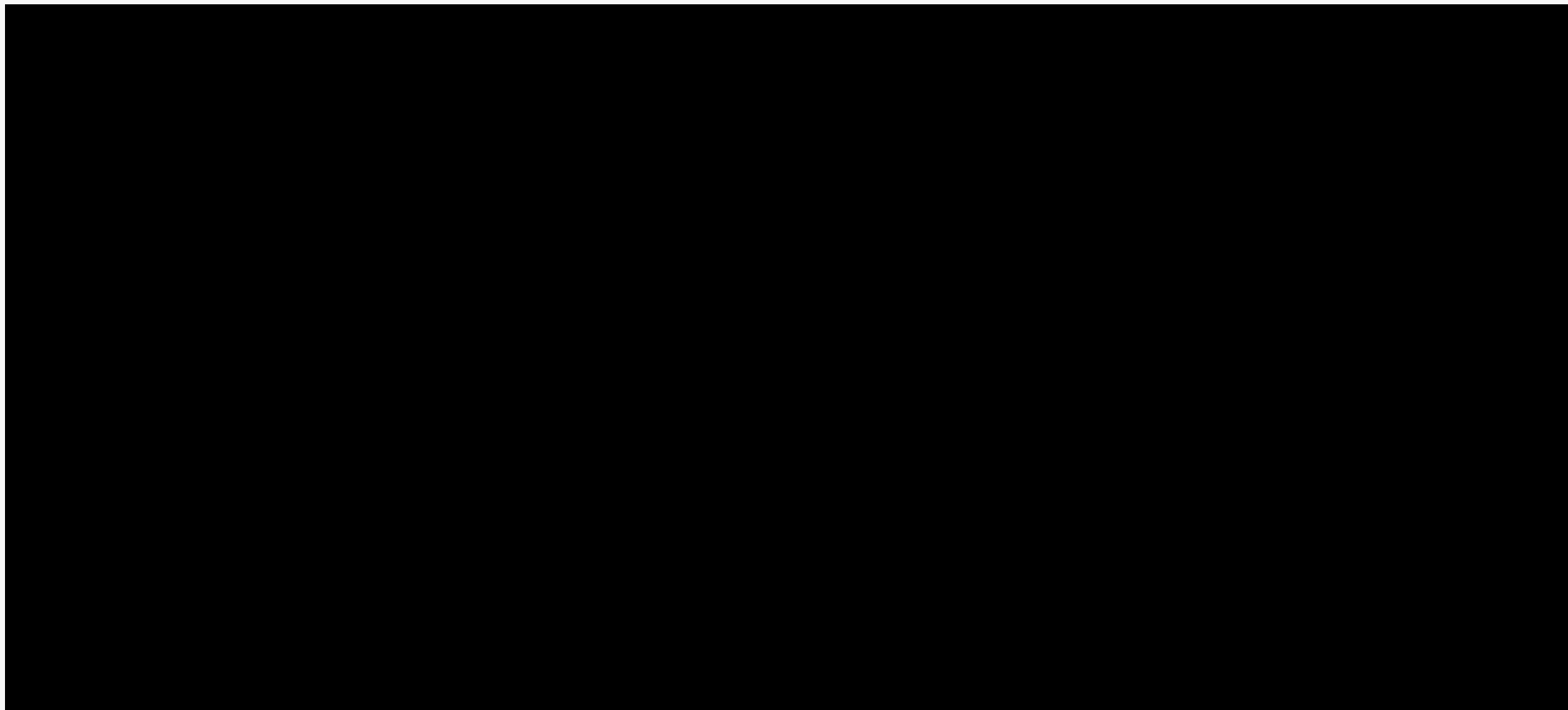


Real-time AI describes AI operations that must guarantee response times within a specified time (deadline), usually a relatively short time.

Where real-time AI is needed



Example: Realtime Super Resolution (by CoCoPIE Inc.)



High-performance computing (HPC)

uses supercomputers or computer clusters to solve advanced computation problems that are usually computation-intensive.



High-performance Machine Learning

uses supercomputers or computer clusters to solve machine learning problems that are computation-intensive.



175 billion parameters

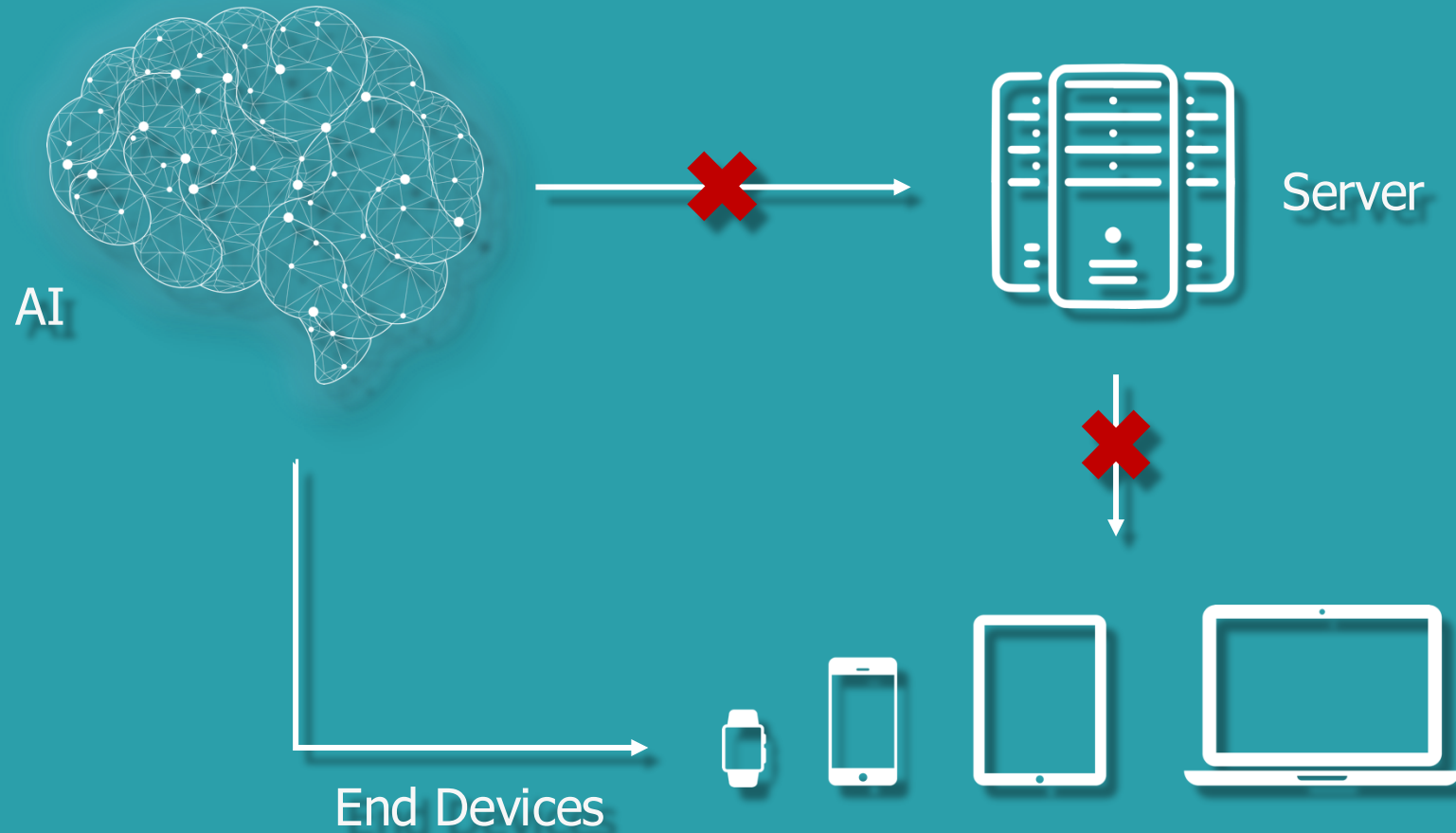
36 years to train on 8
V100 GPUs



1.7 Trillion parameters

Why is this topic important?

AI Trend: From Cloud to Device



AI Trend: From Cloud to Device

Privacy

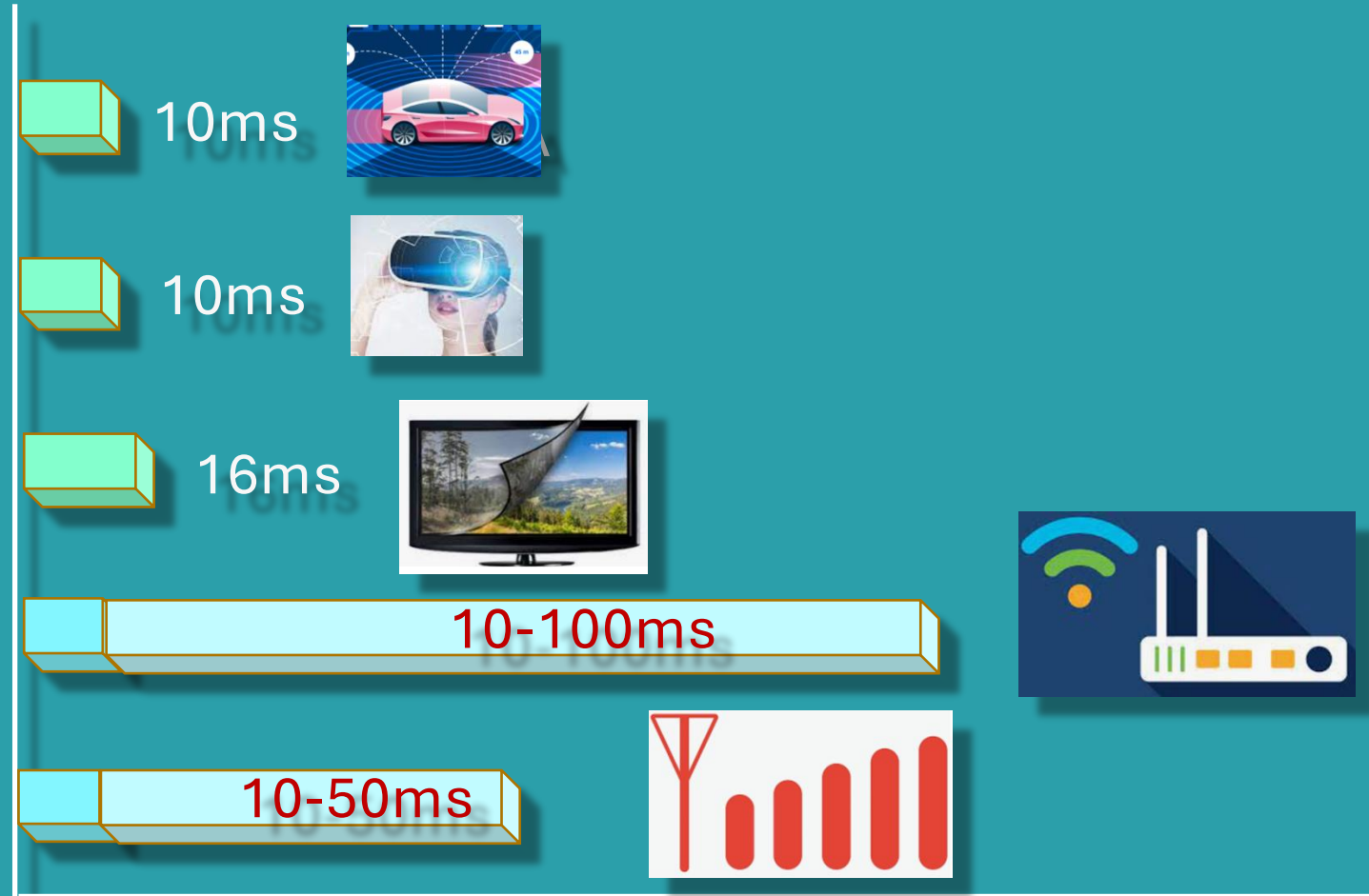


THE STAKES COULDN'T BE HIGHER FOR FACEBOOK TO GET THIS RIGHT Graham Mudd, VP, Facebook

Source: The Verge

AI Trend: From Cloud to Device

Real-Time



AI Trend: From Cloud to Device

Cost

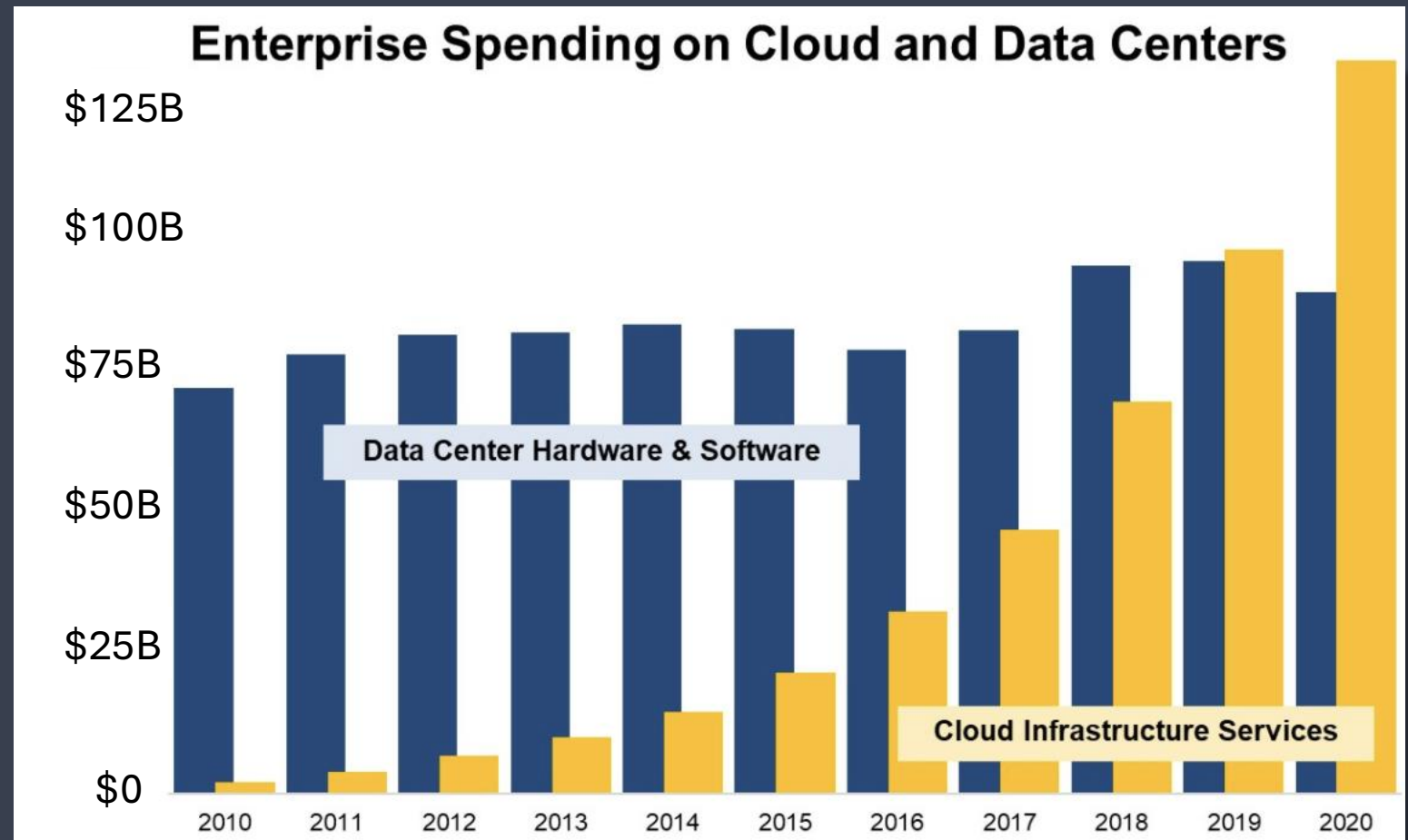


Image Credits: Synergy Research Group

Billions Market

Service providers



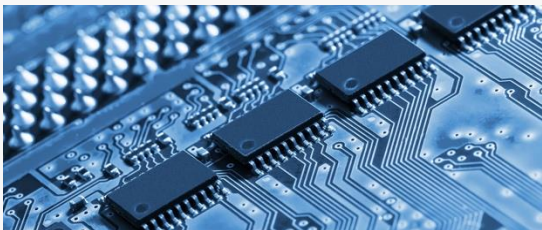
- Media processing and enhancement
- AR/VR, NLP, etc.
- Personalization and recommendation
- On-device training

Device makers



- Smart wearables, smart phone
- Automobile, drone, robots
- smart home, smart city
- Smart manufacturing and farming

Chip makers



- MCU, CPU, GPU, DSP, etc.
- Chips for camera, connectivity, storage, sensors, etc.
- AI and acceleration chips

Consumer Electronics



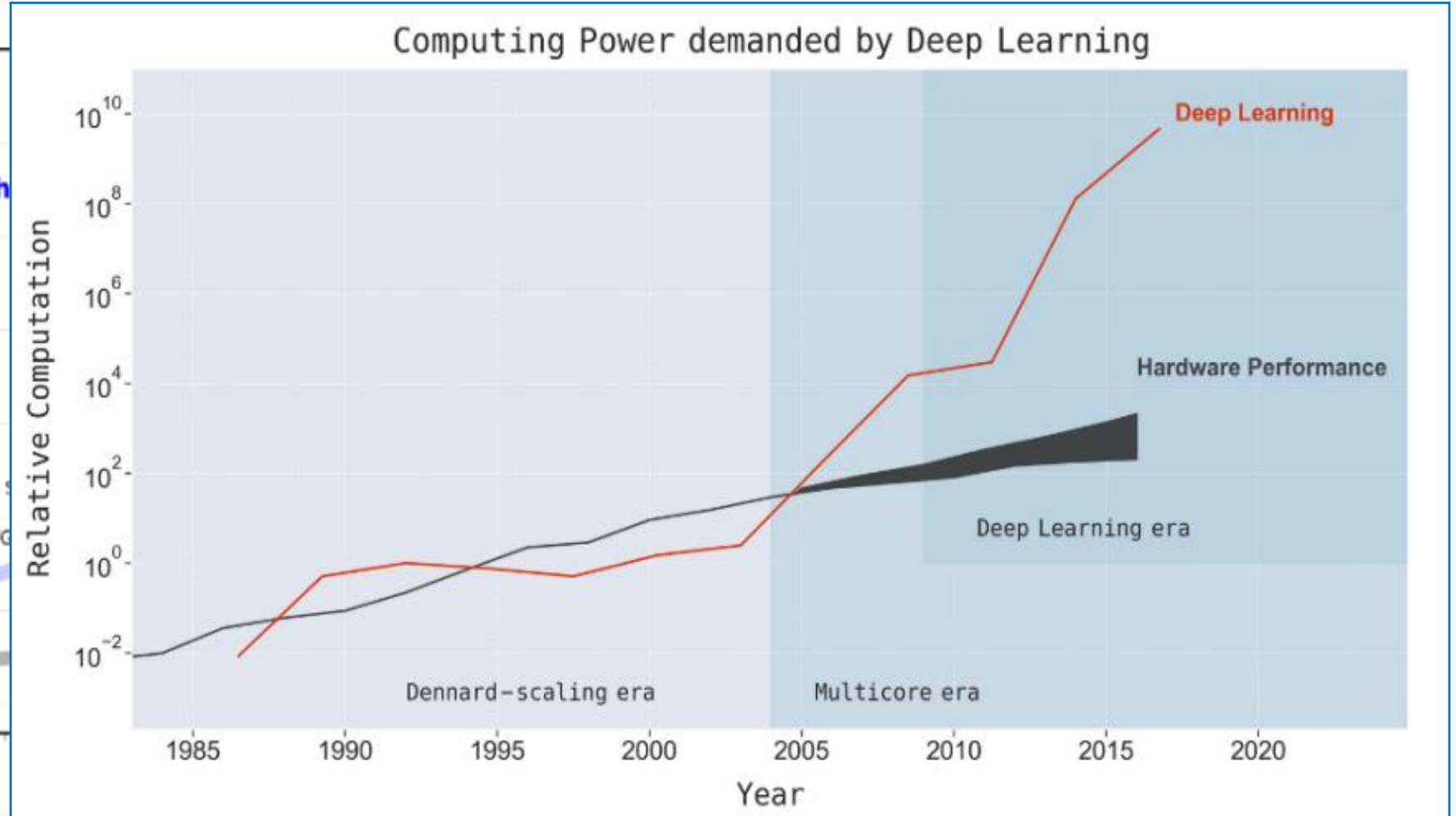
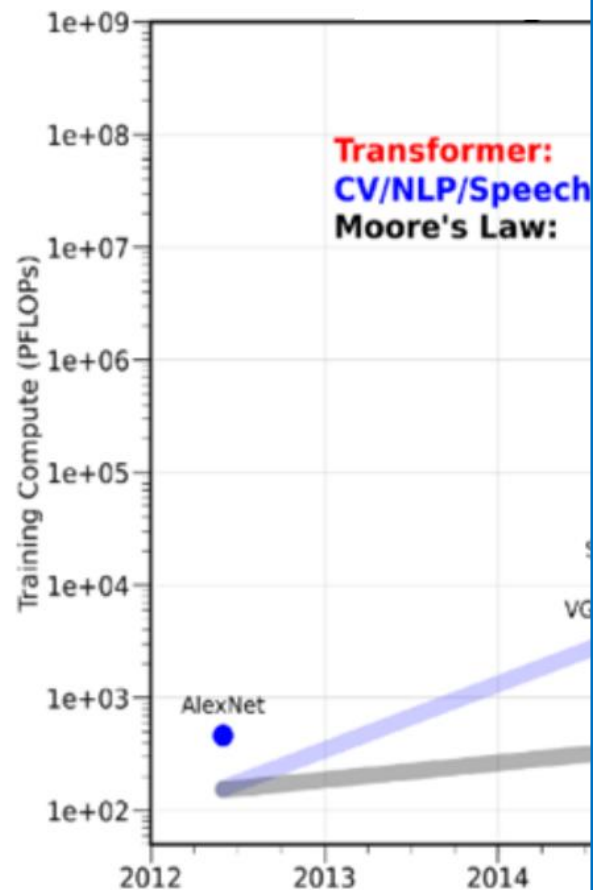
Intelligent Embedded



Industry and Farms



Expanding Gap

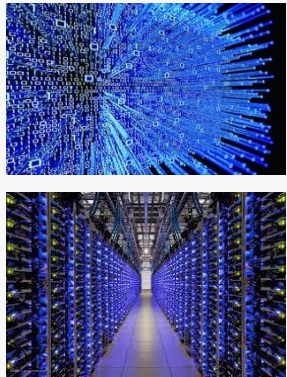




What is the current status?

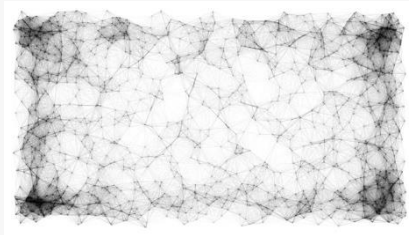
Main stages

Data & Servers



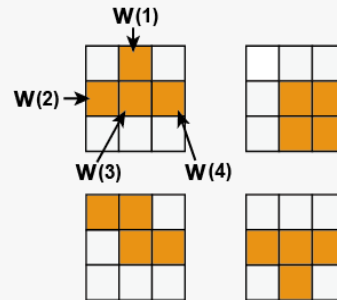
Training phase
(server-side training)

1 Model Training



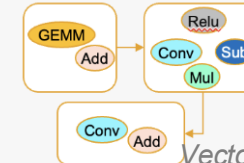
Development phase
(off-device optimization)

2 Model Optimization



ML Systems

3 Code Optimizations



Vector code generation:
For (i=0; i<25; i++)
Calculate multiplication of vectors:
<a[4i]; a[4i+1]; a[4i+2]; a[4i+3]>
*<b[4i]; b[4i+1]; b[4i+2]; b[4i+3]>

Deployment phase
(On-device execution)

4 Runtime Optimization



5 Hardware Improvement

Active research in every stage.

Stage 1: Model Training

How to train faster?

How to achieve the highest accuracy?

How to deal with memory constraints of hardware?

- Distributed training frameworks
- Optimizers for backward propagation
- Memory optimizations
- Hyper parameter tuning

Stage 2: Model Optimization

How to reduce the size of a model while keeping it accurate?

- Model pruning
- Quantization
- Neural Architecture Search
- Distillation

32 bit	32 bit	32 bit	32 bit
32 bit	32 bit	32 bit	32 bit
32 bit	32 bit	32 bit	32 bit
32 bit	32 bit	32 bit	32 bit

Pruning

32 bit	32 bit
32 bit	32 bit

32 bit	32 bit	32 bit	32 bit
32 bit	32 bit	32 bit	32 bit
32 bit	32 bit	32 bit	32 bit
32 bit	32 bit	32 bit	32 bit

Quantization

8 bit	8 bit	8 bit	8 bit
8 bit	8 bit	8 bit	8 bit
8 bit	8 bit	8 bit	8 bit
8 bit	8 bit	8 bit	8 bit

Stage 3: Code Optimization

How to generate efficient code for a DNN?

- AI Compilation
 - Computation graph optimization
 - Parallelization & vectorization
 - Loop optimization
 - Data layout transformation
 - Kernel fusion
- Reuse of computation results

Stage 3: Code Optimization

API Fusion: in computer-vision pipelines, separate API calls (e.g., detection → segmentation → caption) repeat redundant feature extraction. Fusing them gives $\sim X\times$ speed-up without accuracy loss. *(insert 1 motivation figure from your paper — e.g., the "separate vs. fused" diagram)*

Stage 4: Runtime Optimization

How to run a DNN efficiently?

- Heterogeneous scheduling
- Runtime adaptation

Stage 5: Hardware Improvement

How to make hardware best support DNNs?

- Neural Processing Units
 - Special architecture designs
- DNN-oriented features on GPUs
 - Special instructions
 - Special support of certain Tensor patterns